



Ansible Jinja2模板管理

誉天教育

学习目标

- Jinja2模板
- template模块
- Jinja2过滤器



- Jinja2是基于python的模板引擎，那么什么是模板？
- 案例：假设说现在我们需要一次性在10台主机上安装redis，这个通过playbook现在已经很容易实现。默认情况下，所有的redis安装完成之后，我们可以统一为其分发配置文件。这个时候就面临一个问题，这些redis需要监听的地址各不相同，我们也不可能为每一个redis单独写一个配置文件。因为这些配置文件中，绝大部分的配置其实都是相同的。这个时候最好的方式其实就是用一个通用的配置文件来解决所有的问题。将所有需要修改的地方使用变量替换，如下示例中redis.conf.j2文件：

redis.conf.j2文件

修改redis的配置文件如下：

```
bind {{ ansible_ens160.ipv4.address }} 127.0.0.1
```

将其作为模板文件发送到被控端。

- name: copy redis.conf to dest

template:

src: redis.conf.j2

dest: /etc/redis.conf

- playbook使用template模块来实现模板文件的分发，其用法与copy模块基本相同，唯一的区别是，copy模块会将原文件原封不动的复制到被控端，而template会将原文件复制到被控端，并且使用变量的值将文件中的变量替换以生成完整的配置文件。
- 关于template模块的更多参数说明：
 - ◆ src：源模板文件路径
 - ◆ dest：目标文件路径
 - ◆ group：目标文件属组
 - ◆ mode：目标文件的权限
 - ◆ owner：目标文件属主
 - ◆ force：是否强制覆盖，默认为yes
 - ◆ backup：如果原目标文件存在，则先备份目标文件

- 我们直接取了被控节点的eth0网卡的ip作为其监听地址。那么假如有些机器的网卡是bond0，这种做法就会报错。这个时候我们就需要在模板文件中定义条件语句如下：

```
{% if ansible_bond0 is defined %}  
bind {{ ansible_bond0.ipv4.address }} 127.0.0.1  
{% elif ansible_ens160 is defined %}  
bind {{ ansible_ens160.ipv4.address }} 127.0.0.1  
{% else %}  
bind 0.0.0.0  
{% endif %}
```

- 我们可以更进一步，让redis主从角色都可以使用该文件：

```
{% if masterip is defined %}
```

```
slaveof {{ masterip }} {{ masterport|default(6379) }}
```

```
{% endif %}
```

定义一个inventory如下：

```
[redis]
```

```
192.168.40.10
```

```
192.168.40.11 masterip=192.168.40.10 masterport=9999
```

- 定义一个inventory示例如下：

```
[proxy]
192.168.40.10

[webserver]
192.168.40.11
192.168.40.12
```

- 现在把proxy主机组中的主机作为代理服务器，安装nginx做反向代理，将请求转发至后面的两台webserver，即webserver组的服务器。

- proxy主机组中nginx的配置文件如下：

```
upstream web {  
    {% for host in groups['webservers'] %}  
        {% if hostvars[host]['ansible_bond0'] is defined %}  
            server {{ hostvars[host]['ansible_bond0'] }}:80 ;  
        {% elif hostvars[host]['ansible_ens160'] is defined %}  
            server {{ hostvars[host]['ansible_ens160']['ipv4']['address'] }}:80;  
        {% endif %}  
    {% endfor %}  
}  
  
server {  
    listen      80 default_server;  
    server_name _;  
    location / {  
        proxy_pass http://web;  
    }  
}
```

1. default过滤器

当指定的变量不存在时，用于设定默认值

```
"Host": "{{ db_host | default('lcoalhost') }}"
```

案例：

```
- hosts: test
vars:
  paths:
    - path: /tmp/test
      mode: '0400'
    - path: /tmp/foo
    - path: /tmp/bar
tasks:
  - file:
      path: "{{ item.path }}"
      state: touch
      mode: "{{ item.mode|default(omit) }}"
      with_items: "{{ paths }}"
```

2. 字符串操作相关的过滤器

upper : 将所有字符串转换为大写

lower : 将所有字符串转换为小写

capitalize : 将字符串的首字母大写，其他字母小写

reverse : 将字符串倒序排列

first : 返回字符串的第一个字符

last : 返回字符串的最后一个字符

trim : 将字符串开头和结尾的空格去掉

center(30) : 将字符串放在中间，并且字符串两边用空格补齐30位

length : 返回字符串的长度，与count等价

list : 将字符串转换为列表

shuffle : list将字符串转换为列表，但是顺序排列，shuffle同样将字符串转换为列表，但是会随机打乱字符串顺序

2. 示例

```
- hosts: test
gather_facts: no
vars:
  teststr: "abc123ABV"
  teststr1: " abc "
  teststr2: "123456789"
  teststr3: "sfacb1335@#$$%"
tasks:
  - debug:
      msg: "{{ teststr | upper }}"
  - debug:
      msg: "{{ teststr | lower }}"
  - debug:
      msg: "{{ teststr | capitalize }}"
  - debug:
      msg: "{{ teststr | reverse }}"
  - debug:
      msg: "{{ teststr|first }}"
  - debug:
      msg: "{{ teststr|last }}"
```

3. 数字操作相关的过滤器

int : 将对应的值转换为整数

float : 将对应的值转换为浮点数

abs : 获取绝对值

round : 小数点四舍五入

random : 从一个给定的范围中获取随机值

3. 示例

```
- hosts: test
gather_facts: no
vars:
  testnum: -1
tasks:
  - debug:
      msg: "{{ 8+('8'|int) }}"
  - debug:
      # 默认情况下, 如果无法完成数字转换则返回0
      # 这里指定如果无法完成数字转换则返回6
      msg: "{{ 'a'|int(default=6) }}"
  - debug:
      msg: "{{ '8'|float }}"
  - debug:
      msg: "{{ 'a'|float(8.88) }}"
  - debug:
      msg: "{{ testnum|abs }}"
  - debug:
      msg: "{{ 12.5|round }}"
  - debug:
      msg: "{{ 3.1415926 | round(5) }}"
  - debug:
      # 从0到100中随机返回一个数字
      msg: "{{ 100|random }}"
  - debug:
      # 从5到10中随机返回一个数字
      msg: "{{ 10|random(start=5) }}"
  - debug:
      # 从4到15中随机返回一个数字, 步长为3
      # 返回的随机数只可能是: 4, 7, 10, 13中的一个
      msg: "{{ 15|random(start=4,step=3) }}"
  - debug:
      # 从0到15随机返回一个数字, 步长为4
      msg: "{{ 15|random(step=4) }}"
```

4. 列表操作相关的过滤器数字操作相关的过滤器

length: 返回列表长度

first: 返回列表的第一个值

last: 返回列表的最后一个值

min: 返回列表中最小的值

max: 返回列表中最大的值

sort: 重新排列列表, 默认为升序排列, sort(reverse=true)为降序

sum: 返回纯数字非嵌套列表中所有数字的和

flatten: 如果列表中包含列表, 则flatten可拉平嵌套的列表, levels参数可用于指定被拉平的层级

join: 将列表中的元素合并为一个字符串

random: 从列表中随机返回一个元素

union: 将两个列表合并, 如果元素有重复, 则只留下一个

intersect: 获取两个列表的交集

difference: 获取存在于第一个列表中, 但不存在于第二个列表中的元素

symmetric_difference: 取出两个列表中各自独立的元素, 如果重复则只留一个

4. 列表操作相关的过滤器数字操作相关的过滤器

```
- hosts: test
gather_facts: false
vars:
  testlist1: [1,2,4,6,3,5]
  testlist2: [1,[2,3,4,[5,6]]]
  testlist3: [1,2,'a','b']
  testlist4: [1,'A','b',['C','d'],'Efg']
  testlist5: ['abc',1,2,'a',3,2,'1','abc']
  testlist6: ['abc',3,'1','b','a']
tasks:
  - debug:
      msg: "{{ testlist1 | length }}"
  - debug:
      msg: "{{ testlist1 |first }}"
  - debug:
      msg: "{{ testlist1 | last }}"
  - debug:
      msg: "{{ testlist1 | min }}"
  - debug:
      msg: "{{ testlist1 | max }}"
  - debug:
      msg: "{{ testlist1 | sort }}"
  - debug:
      msg: "{{ testlist1 | sort(reverse=true) }}"
  - debug:
      msg: "{{ testlist2 | flatten | max }}"
  - debug:
      msg: "{{ testlist4 | upper }}"
  - debug:
      msg: "{{ testlist4 | lower }}"
  - debug:
      msg: "{{ testlist5 | union(testlist6) }}"
  - debug:
      msg: "{{ testlist5 | intersect(testlist6) }}"
  - debug:
      msg: "{{ testlist5 | difference(testlist6) }}"
```


- ✓ Jinja2模板
- ✓ template模块
- ✓ Jinja2过滤器

